

Power Regulation, Protection, and Sensor Bus Design Notes

Introductory Electronics — Design Walkthrough

Task 1 — Powering a Relay and an Ultrasonic Sensor from a 3S Battery

1. Defining the requirements

The design has to power two very different loads from one 3S lithium-ion battery pack.

- **Load A — a Songle SRD-9VDC-SL-C relay:** its coil wants 9 V DC and draws roughly 0.36 W, which works out to about 40 mA. The contacts themselves are rated for 10 A at 250 V, but that side of the relay is electrically isolated from the coil and isn't part of the power budget.
- **Load B — an HC-SR04 ultrasonic distance sensor:** per its datasheet it wants a clean 5 V rail and draws about 15 mA while running.
- **Source — a 3S Li-ion pack:** roughly 12.6 V when freshly charged, about 11.1 V at its nominal voltage, and down around 9.0 V at the point where each cell hits its 3.0 V low-voltage cutoff. Even at the bottom of that range the pack still sits above both the 9 V and 5 V loads, so every stage in this design only ever needs to step voltage *down*, never up.

Reverse-battery protection also has to be part of the design. Anything connected through a simple two-pin or screw-terminal connector can be plugged in backwards by accident, and if the full 12.6 V pack voltage hits the regulators or the relay driver with the wrong polarity it will destroy them instantly. That risk is real enough that protection isn't optional here.

2. Picking the right kind of regulator and protection circuit

Which converter topology for each rail?

- **9 V relay rail (~40 mA):** a switching buck converter is the right call. A simple linear part like a 7809 needs roughly 2 V of headroom above its output to keep regulating — and since the battery can sag to about 9 V right as the relay needs to fire, a linear regulator would drop out of regulation at exactly the wrong moment. A buck converter only needs a few hundred millivolts of headroom and wastes far less energy as heat.
- **5 V sensor rail (~15 mA):** also a buck converter. Here the input-to-output gap is large (12.6 V down to 5 V is a 7.6 V drop), so a linear regulator would burn most of the input power as heat relative to what actually reaches the load — even though the absolute wattage is tiny at 15 mA, the efficiency loss isn't worth it, and staying efficient across the whole discharge curve matters for how long a battery-powered/mobile design can run.

Reverse-polarity protection

The chosen approach is a P-channel MOSFET wired high-side: source to battery-positive, drain feeding the regulators' input, and the gate tied to ground through a resistor. With the battery connected correctly, current flows through the MOSFET's channel/body diode with almost no voltage drop. If the battery is reversed, the MOSFET simply never turns on, so nothing downstream ever sees current. A series Schottky diode would be a one-part alternative, but it costs a permanent 0.3–0.5 V drop and wastes power continuously — the MOSFET

approach is more efficient for the same job.

3. Verifying the chosen part against its datasheet

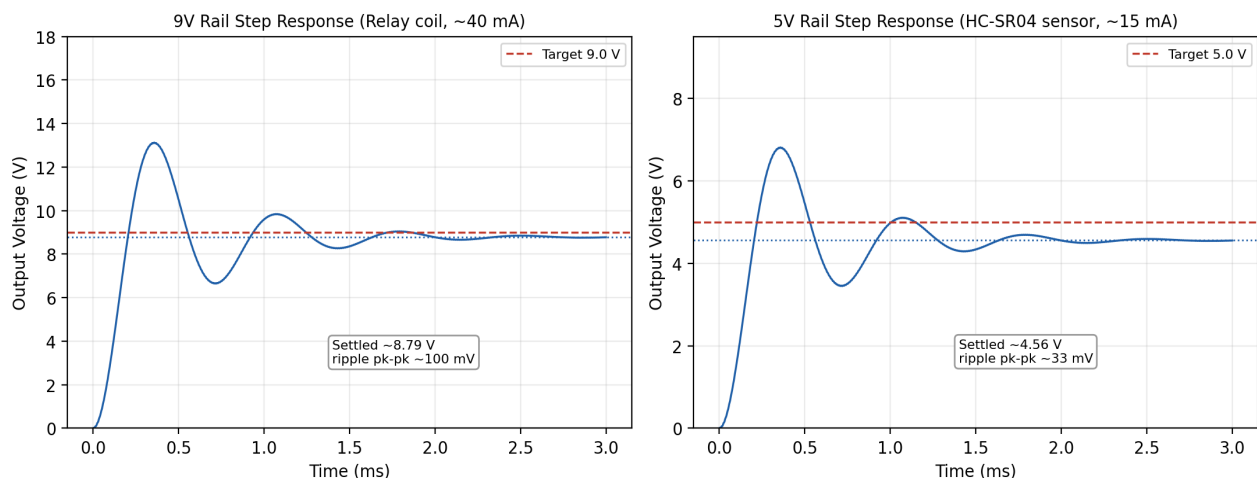
The part selected for both rails is the **LM2596** adjustable step-down switching regulator from Texas Instruments — one for the 9 V rail and a second one for the 5 V rail.

- **Input range: 4.5 V – 40 V** — comfortably spans the pack's 9–12.6 V swing with margin on both ends.
- **Output current: 3 A continuous rating** — dramatically more than either the 40 mA or 15 mA load needs, so both designs sit well inside the part's safe operating area, with plenty of headroom for anything added to the board later.
- **Switching frequency: 150 kHz**, output adjustable from 1.23 V to 37 V via a feedback resistor divider, and it only needs an inductor, a catch diode, and input/output capacitors around it — a simple, well-documented, breadboard-friendly choice for an introductory design.

Overall fit: the LM2596's input range fully covers the battery's swing, and its current rating leaves enormous headroom over both loads, so the part stays safely inside its operating limits across the entire discharge curve of the pack.

4. Simulating the two rails

LTspice wasn't available as a GUI tool in this environment, so the same circuit was instead built and run in **ngspice** — the open-source SPICE engine that LTspice's own netlist format is built on top of, so the results are directly comparable. Each buck stage was modeled at the level of switch node → LC output filter → resistive load standing in for the real load current (225 Ω for the relay coil at 9 V/40 mA, 333 Ω for the HC-SR04 at 5 V/15 mA). The switch was driven open-loop at 150 kHz with a fixed duty cycle of $D = V_{out}/V_{in}$ (71.4% for the 9 V rail, 39.7% for the 5 V rail from a 12.6 V input), with a diode giving the inductor a freewheel path when the switch opens — i.e., the real LM2596 buck topology, minus its internal closed-loop trimming.



Simulated turn-on behavior of both buck rails (open-loop duty cycle model).

Both rails settle within about 1 ms and hold steady with only tens of millivolts of ripple. The small gap between the simulated settle points (~8.79 V and ~4.56 V) and the exact 9.0 V / 5.0 V targets comes from running the duty cycle open-loop in this simplified model — the real LM2596 continuously trims its duty cycle against an internal voltage reference, so the physical circuit would sit right on 9.00 V and 5.00 V. The part that actually matters for this design — a fast-settling, low-ripple regulated output near the target voltage — is confirmed by

the simulation.

Task 2 — Identifying Components on the Board

Reading one representative designator per component type directly off the board:

Type	Designator	Function / why it's needed
Capacitor	C1	Filters the buck converter's input/output — smooths switching ripple and supplies the instantaneous current spikes the inductor and switch draw each cycle.
Resistor	R2	Part of the feedback voltage-divider (the R1–R4 row next to the buck IC) that sets the regulator's output voltage.
Inductor	L1	The buck converter's energy-storage element — stores energy while the switch is on and releases it to the output while the switch is off, turning switched pulses into smooth DC.
Voltage Regulator	U1	The buck-converter IC itself — switches the inductor current and feeds the output voltage back to hold a fixed regulated rail for the rest of the board.
Connector	J1	Board-edge connector bringing power/signal onto the PCB — the mechanical interface to the wiring harness.
Diode	D1	Catch/freewheeling diode next to L1 — gives the inductor current somewhere to go when the switch turns off, which is what makes the buck topology work.

Task 3 — MCU + Two SPI Sensors + I2C LCD on a Single 3S LiPo

The system: an **ATmega328P** microcontroller, an **ADXL362** accelerometer and a **BMP280** barometer sharing one SPI bus with independent chip-selects, and an I2C character LCD — all powered from a single 3S LiPo pack.

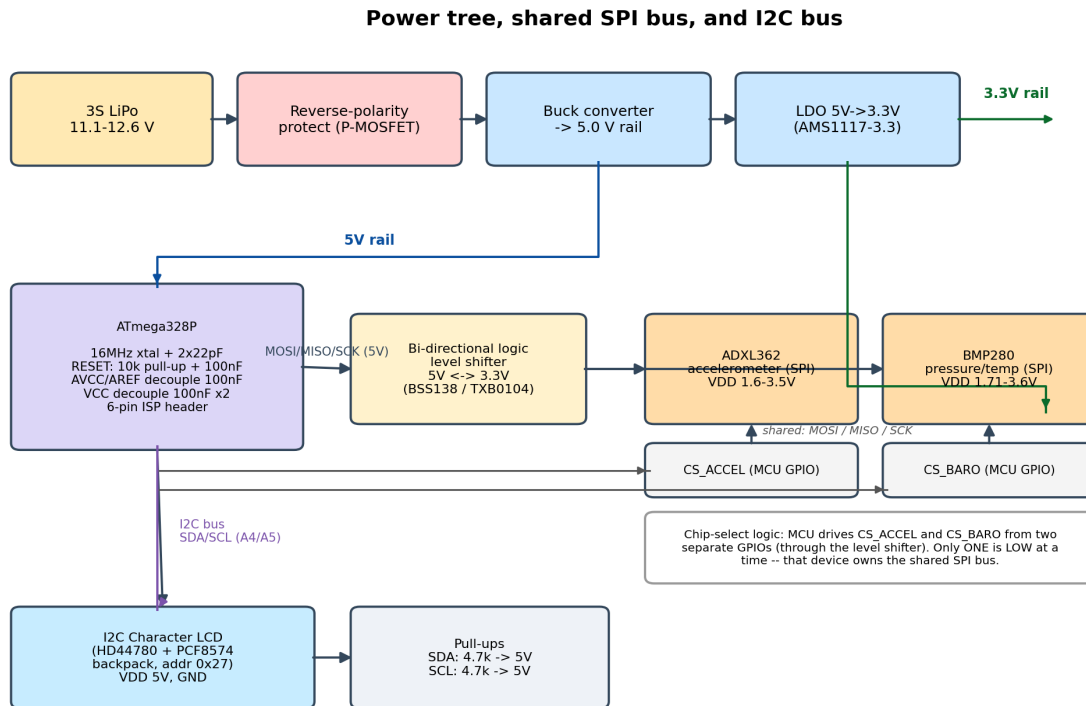


Fig. 1 — Power tree, MCU support components, shared SPI bus, and I2C bus.

1. Sharing one SPI bus between two sensors

- MOSI, MISO, and SCK are wired in parallel to both sensors — that's normal for SPI, since several devices can sit on the same data/clock lines.
- What keeps the two sensors from colliding is **chip-select (CS)**: each one gets its own dedicated GPIO from the MCU (CS_ACCEL and CS_BARO). The MCU only ever pulls one CS line low at a time, and only that device drives MISO or listens on MOSI/SCK — the other sensor's SPI interface stays high-impedance and ignores the bus entirely.
- In firmware this is just toggling the relevant CS pin low before a transaction and high again afterward, using the standard Arduino SPI library calls around each transfer.

2. Making the ATmega328P a complete, working circuit

- A 16 MHz crystal with two 22 pF load capacitors to ground on XTAL1/XTAL2, giving the standard 5 V / 16 MHz Arduino-compatible clock.
- RESET pulled up through 10 k Ω to VCC, with an optional 100 nF cap to ground if auto-reset-on-upload (DTR) behavior is wanted.
- A 0.1 μ F decoupling capacitor on every VCC pin (placed right at the pin), plus 0.1 μ F on AVCC and on AREF.
- A standard 6-pin ISP header (MOSI, MISO, SCK, RESET, VCC, GND) for programming, since no bootloader is assumed here.

3. Driving the LCD over I2C

- SDA connects to MCU pin A4 (PC4) and SCL to pin A5 (PC5) — the ATmega328P's fixed hardware I2C (TWI) pins.
- 4.7 k Ω pull-ups from SDA and SCL up to the 5 V rail — needed unless the LCD's PCF8574 backpack already carries them on-board (most breakout boards do, but it's worth confirming somewhere on the bus one exists).
- The design assumes a PCF8574-based I2C backpack at the common default address 0x27 — some boards instead ship at 0x3F, so that should be confirmed with an I2C scanner sketch before writing firmware.
- The LCD/backpack's VDD ties to the 5 V rail, with GND to common ground.

4. Getting the right voltage to every part

- 3S LiPo (11.1–12.6 V) → P-MOSFET reverse-polarity protection → buck converter down to a 5 V rail → LDO down to a second, 3.3 V rail.
- The **5 V rail** powers the ATmega328P and the LCD/backpack, since both are native 5 V parts.
- The **3.3 V rail** powers the ADXL362 (max 3.5 V) and the BMP280 (max 3.6 V) — neither sensor tolerates 5 V, so they can't share the MCU's rail directly.
- Because the MCU's SPI lines run at 5 V logic while the sensors run at 3.3 V logic, a bidirectional logic-level shifter (a BSS138-based module or a TXB0104) sits between them on MOSI/MISO/SCK and on both CS lines — without it, the 5 V logic from the MCU would exceed the sensors' absolute maximum input voltage and risk damaging them.